COMP1003
Algorithms, Data Structures and Mathematics

20 CREDIT MODULE

ASSESSMENT: 100% Coursework **W1: 30% Set Exercises**
W2: 70% Report

MODULE LEADER: Thomas Wennekers

MODULE AIMS

- To give students a basic understanding of algorithmic design and practice
- To give students a basic understanding of mathematical principles underlying Computing

ASSESSED LEARNING OUTCOMES (ALO):

1. Explain the importance of algorithmic design in optimizing use of computing resources.
2. Review fundamental mathematical and logical principles that underlie modern computer science.
3. Identify suitable structures and algorithms to implement programming tasks.
4. Synthesize the solution to a real-world task as a combination of two or more standard algorithms.

Overview

This document contains all the necessary information pertaining to the assessment of *COMP1003 Algorithms, Data Structures and Mathematics*. The module is assessed via **100% coursework**, across two elements: *30% Set Exercises* and *70% Report*.

The sections that follow will detail the assessment tasks that are to be undertaken. The submission and expected feedback dates are presented in Table 1. All assessments are to be submitted electronically via the respective DLE module pages before the stated deadlines.

	Submission Deadline	Feedback
Set Exercises (30%)	**21/03/22 4pm**	25/04/22
Report (70%)	**16/05/22 4pm**	15/06/22

Table 1: Assessment Deadlines

Whilst the assessment information is provided at the start of the module, it is not necessarily expected you will start this immediately – as you will often not have sufficient understanding of the topic. The module leader will provide guidance in this respect.

Assessment 1: Set Exercises

Answer the following questions. Hand-in a single PDF file that contains solutions to the questions in the same order they appear below. Solutions must show your working and have explanations. Highlight final results of calculations, for example, by underlining them. You can submit type-set answers (for example using LaTeX or similar) or hand-written answers. The latter may be scanned or photographed. If you submit hand-written answers, make sure they are tidy and well readable.

Q1 Mathematical Induction

[Total 10 marks]

Prove using mathematical induction that $2^n > n$ for all n in **N**.
What is the limit of $2^n/n$ if n grows towards infinity, and what if it goes towards 0?
What is the limit of $(n+1)/2^n$ if n grows towards infinity?
Explain your answers.

Q2 Functions

[Total 20 marks]

Consider the function $f : \mathbf{R} \rightarrow \mathbf{R}$, $f(x) = (x-1)^2(x+1)$

a) Turn f into a polynomial.

[2 marks]

b) Compute the derivative and second derivative of f.

[4 marks]

c) Compute the zero crossings, minima and maxima of f (if any).

[4 marks]

d) Compute the indefinite integral of f.

[3 marks]

e) Compute the integral of f between -2 and 2.

[2 marks]

f) Determine the limits of f(x) when x approaches +infinity and –infinity.

[2 marks]

g) Draw the graph of f indicating the zero crossings, minima, maxima and limits.

[3 marks]

Q3 Order and Metric

[Total 20 marks]

a) Prove that divisibility over the natural numbers induces a partial order.
(Hint: You must proof that the axioms of a (partial) order hold.)

[10 marks]

b) Prove that the lexicographic ordering on alphanumeric strings is an order or disprove this if it is not true.

[10 marks]

Q4 Probability

[Total 20 marks]

A sample space consists of 9 elementary outcomes e_1, e_2, …, e_9 whose probabilities are $P(e_1) = P(e_2) = 0.07$, $P(e_3) = P(e_4) = P(e_5) = 0.1$, $P(e_6) = P(e_7) = 0.2$, $P(e_8) = P(e_9) = 0.08$

Suppose A = {e1, e5, e8}, B = {e2, e5, e8, e9}

a) Compute the Union, Intersection, Set difference A/B and B/A, and the Symmetric Difference between A and B.

[5 marks]

b) Calculate P(A), P(B) and P(A ∩ B)

[3 marks]

c) Using the addition law of probability, calculate P(A U B)

[3 marks]

d) List the composition of the event A U B, and calculate P(A U B) by adding the probabilities of the elementary outcomes.

[3 marks]

e) Calculate P(B') from P(B), also calculate P(B') directly from the elementary outcomes of B'.

[6 marks]

Threshold Criteria

To achieve a pass (40%+), 40% of the total marks for this CW element

To achieve a merit (60%+), 60% of the total marks for this CW element

To achieve a distinction (70%+), 70% of the total marks for this CW element

Assessment 2: “Report”

Task: Programming exercise

Write a console program (using Console.WriteLine and ReadLine and no GUI) to allow a user to do conversions between two different units of measurement. The user might type in an amount and the names of two units of measurement, for example (note the format):

5, ounce, gram

In this case the program should print out the number of grams equal to 5 ounces. Because 1 ounce is 28 grams it should print:

5 ounces are 140 grams

Unit names and conversion factors will be supplied as a file convert.txt, and this should be read in once by the program at start-up. The file consists of a series of lines, each consisting of two unit names (both strings) and a conversion factor (a number with a decimal point) separated by commas, blanks or semi-colons, for example:

ounce,gram,28.0

This line means that one ounce equals 28.0 grams. There may be blanks (as in "gram , ounce, 0.036") in the file. A sample file, convert.txt, is available on the portal (included as a figure below), but your program should be able to cope with **any** file laid out in this way. You will need “System.IO” in the header of the program for methods that open a file and read from it. You are allowed to use a StreamReader and may only go through the data in the file a single time using the method ReadLine. The use of Split and Length are allowed. ToDouble, ToInt, ToString are allowed.

If the user types in an incompatible pair of units (such as 5, gram, pint) the program should take appropriate action. Your program should repeatedly prompt the user for a new input. If a positive amount is typed in, the program should proceed with the conversion; when a zero or negative amount is typed the program should halt.

Example Input file: convert.txt

convert.txt

```
1  ounce,gram,28.0
2  pound,ounce,16.0
3  pound,kilogram,0.454
4  pint,litre,0.568
5
6  inch,centimetre,2.5
7  mile, Inch, 63360.0
8
```

Format of coursework to be handed in:

Provide **a single C#** file containing the functions/methods/sub-routines requested in the task defined above including a Main program that tests the routines to convince the user/programmer that the code is correct. Don’t submit a complete C# solution. Also provide a screenshot or other printout of

your running program while the user requests 3 different conversions and then exits. Zip both files up before you upload them on the DLE page of the module.

Assessment Criteria:

Marks will be provided according to the following main categories

1. General Code properties (30%)
   This category refers mainly to good programming style. More specifically, aspects considered will be: Proper modularisation of the algorithmic structure; the correct use of data structures; originality of the solution; correctness and completeness; appropriate names of identifiers and methods; code readability; consistent layout and indentation; good commenting; error/exception handling. Note, that C#-instrinsic data structures from the Collections libraries like Lists, Stacks or ArrayLists must not be used!

2. Task Specific Code Properties (70%)
   There are many ways to solve the task. You should use data structures and algorithms that are best suited to solve the task. All structures and algorithms used should be as efficient as possible assuming a user may want to request many conversions. More advanced or efficient solutions will attract higher marks. All required code for functionality requested in the task specification should be implemented by the student unless mentioned otherwise. Functionality not asked for will not attract additional marks but may lead to subtracted marks if it interferes with the code quality (for example, the General Code properties). The code should be safe to use and not crash.

Threshold Criteria:

To achieve a pass (40%+), 40% of the total marks for this CW element

To achieve a merit (60%+), 60% of the total marks for this CW element

To achieve a distinction (70%+), 70% of the total marks for this CW element

General Guidance

Extenuating Circumstances

There may be a time during this module where you experience a serious situation which has a significant impact on your ability to complete the assessments. The definition of these can be found in the University Policy on Extenuating Circumstances here: https://www.plymouth.ac.uk/uploads/production/document/path/7/7741/Extenuating_Circumstances_Policy_and_Procedures.pdf

Students with valid EC's may claim either for an extension period or for non-submission. Valid for Non-Submission means you will be asked to do a new piece of work. Please see below.

Referral

Please note that if you claim for non-submission and are offered a referral, you will be required to complete a NEW piece of work for the module. The new piece of work will assess that you have met the learning outcomes for the module but in a way that will be different to the original set piece. The referral is not a repeat or extension of the original coursework.

Carrying out a new piece of work means you will not be able to keep the marks already gained during the module. Eg: if you pass the set exercises (CW1) but do not submit the main work (CW2) AND are offered a referral, you will not keep the set exercises grade.

Plagiarism

All of your work must be of your own words. You must use references for your sources, however you acquire them. Where you wish to use quotations, these must be a very minor part of your overall work.

To copy another person's work is viewed as plagiarism and is not allowed. Any issues of plagiarism and any form of academic dishonesty are treated very seriously. All your work must be your own and other sources must be identified as being theirs, not yours. The copying of another persons' work could result in a penalty being invoked.

Further information on plagiarism policy can be found here:

Plagiarism: https://www.plymouth.ac.uk/student-life/your-studies/essential-information/regulations/plagiarism

Examination Offences: https://www.plymouth.ac.uk/student-life/your-studies/essential-information/exams/exam-rules-and-regulations/examination-offences

Turnitin (http://www.turnitinuk.com/) is an Internet-based 'originality checking tool' which allows documents to be compared with content on the Internet, in journals and in an archive of previously submitted works. It can help to detect unintentional or deliberate plagiarism.

It is a formative tool that makes it easy for students to review their citations and referencing as an aid to learning good academic practice. Turnitin produces an 'originality report' to help guide you. To learn more about Turnitin go to:
https://guides.turnitin.com/01_Manuals_and_Guides/Student/Student_User_Manual

Referencing

The University of Plymouth Library has produced an online support referencing guide which is available here: http://plymouth.libguides.com/referencing.

Another recommended referencing resource is Cite Them Right Online; this is an online resource which provides you with specific guidance about how to reference lots of different types of materials.

The Learn Higher Network has also provided a number of documents to support students with referencing:
References and Bibliographies Booklet:
http://www.learnhigher.ac.uk/writing-for-university/referencing/references-and-bibliographies-booklet/
Checking your assignments' references:
http://www.learnhigher.ac.uk/writing-for-university/academic-writing/checking-your-assigments-references/